

```

#
# Instructor:      Shawn Smith
# Semester:       Fall 2024
# Course Name:    Calculus I
# Course Number:  2890
# Course Section: 5009
#
# Filename:       FileProcessing.py
# Version:        1.0
# Date Original:  09/22/2024
# Date Modified:  09/25/2024
# Language:       Python
# Description:    See The Design Documents
# Author:         Olis Bahari
#

# Import Module
import csv

# CLASStype
class ClassType:
    def __init__(self, Student_Last, Student_First, Instructor_Last,
Instructor_First, Year, Course_Name, Class_No, Section_No):
        self.Student_Last = Student_Last
        self.Student_First = Student_First
        self.Instructor_Last = Instructor_Last
        self.Instructor_First = Instructor_First
        self.Year = Year
        self.Course_Name = Course_Name
        self.Class_No = Class_No
        self.Section_No = Section_No

# TEST type
class TestType:
    def __init__(self, Section_No, Student_Last, Student_First, Test_Type, Test_No,
Title, Test_Result):
        self.Section_No = Section_No
        self.Student_Last = Student_Last
        self.Student_First = Student_First
        self.Test_Type = Test_Type
        self.Test_No = Test_No
        self.Test_Title = Title
        self.Test_Result = Test_Result

# CFG_SYSTEM type
class Cfg_System_Type:
    def __init__(self, Title, Value, Percent):
        self.Title = Title
        self.Value = Value
        self.Percent = Percent

```

```

# cvs variables
global CLASSReader, TESTreader

# Integer Type
global NumberOfTest, NumberOfRetake, NumberOfLabHomework, NumberOfLabQuiz,
NumberOfFinal

# float type
global TestTotal, LabQuizTotal, LabHomeworkTotal, LabHomeworkAverage
global TestAverage, LabQuizAverage, SemesterAverage, FinalTest
global LabBonusPoint, Test, Grade, SummaryFile, TestArray

global LastName, FirstName

# Set DEBUG True to execute print codes
global DEBUG

# cvs filenames
TEST_FILENAME = "C:\\Dropbox\\Mathematics\\Project\\FileProcessing\\Test.txt"
CLASS_FILENAME = "C:\\Dropbox\\Mathematics\\Project\\FileProcessing\\Class.txt"
SUMMARY_FILENAME = "C:\\Dropbox\\Mathematics\\Project\\FileProcessing\\Summary.txt"

#
# Calculate_Grade_Normal
# Calculate grade A, B, C, D, & F based on the test value
# Input valid test range is 0..100
# Output valid grade range is {A, B, C, D, F}
#
def Calculate_Grade_Normal(Test):

    if (Test >= 90) and (Test <= 100):
        Grade = "A"
    elif (Test >= 80) and (Test < 90):
        Grade = "B"
    elif (Test >= 70) and (Test < 80):
        Grade = "C"
    elif (Test >= 60) and (Test < 70):
        Grade = "D"
    else:
        Grade = "F"

    return (Grade)

#
# Calculate_Grade Plus
# Calculate grade A+,A-,B+,B-,C+,C-,D+,D-, & F based on the test value
# Input valid test range is 0..100
# Output valid grade range is {A+,A-,B+,B-,C+,C-,D+,D-, & F}

```

```

#
def Calculate_Grade_Plus(Test):

    if (Test >= 97) and (Test <= 100):
        Grade = "A+"
    if (Test >= 93) and (Test < 97):
        Grade = "A"
    if (Test >= 90) and (Test < 93):
        Grade = "A-"
    if (Test >= 87) and (Test < 90):
        Grade = "B+"
    if (Test >= 83) and (Test < 87):
        Grade = "B"
    if (Test >= 80) and (Test < 83):
        Grade = "B-"
    if (Test >= 77) and (Test < 80):
        Grade = "C+"
    if (Test >= 73) and (Test < 77):
        Grade = "C"
    if (Test >= 70) and (Test < 73):
        Grade = "C-"
    if (Test >= 67) and (Test < 70):
        Grade = "D+"
    if (Test >= 63) and (Test < 67):
        Grade = "D"
    if (Test >= 60) and (Test < 63):
        Grade = "D-"
    else:
        Grade = "F"

    return (Grade)

```

```

#
# Lab_Bonus_Point
# Lab Bonus Point valid input range is from 0 to 100.
# Lab Bonus Point valid output range is from 0 to 20.
#
def Calculate_Lab_Bonus_Point(Test):
    if (Test >= 90) and (Test <= 100):
        LabBonusPoint = 20
    elif (Test >= 80) and (Test < 90):
        LabBonusPoint = 18
    elif (Test >= 70) and (Test < 80):
        LabBonusPoint = 16
    elif (Test >= 60) and (Test < 70):
        LabBonusPoint = 14
    elif (Test >= 50) and (Test < 60):
        LabBonusPoint = 12
    elif (Test >= 40) and (Test < 50):

```

```

        LabBonusPoint = 10
    elif (Test >= 30) and (Test < 40):
        LabBonusPoint = 8
    elif (Test >= 20) and (Test < 30):
        Bonus_Point =6
    elif (Test >= 10) and (Test < 20):
        LabBonusPoint = 4
    elif (Test >= 5) and (Test < 10):
        LabBonusPoint = 2
    else:
        LabBonusPoint = 0

    return (LabBonusPoint)

```

```

#
# Print_Summary_File_Header
# Print the header for summary or report csv file
#

```

```

def Print_Summary_File_Header():
    SummaryFile.write("SECTION_NO")
    SummaryFile.write(",")
    SummaryFile.write("STUDENT_LAST")
    SummaryFile.write(",")
    SummaryFile.write("STUDENT_FIRST")
    SummaryFile.write(",")
    SummaryFile.write("LAB_HOMEWORK")
    SummaryFile.write(",")
    SummaryFile.write("LAB_BNOUS")
    SummaryFile.write(",")
    SummaryFile.write("LAB_QUIZ_AVERAGE")
    SummaryFile.write(",")
    SummaryFile.write("TEST_AVERAGE")
    SummaryFile.write(",")
    SummaryFile.write("FINAL")
    SummaryFile.write(",")
    SummaryFile.write("SEMESTER_AVERAGE")
    SummaryFile.write(",")
    SummaryFile.write("GRADE")
    SummaryFile.writelines("\n")

```

```

#
# Write data into summary or report cvs file
#
def WriteReportOneStudent(row, LabHomeworkAverage, LabBonusPoint, LabQuizAverage,
TestAverage, FinalTest, SemesterAverage, Grade):
    SummaryFile.write(row['SECTION_NO'])
    SummaryFile.write(",")
    SummaryFile.write(row['STUDENT_LAST'])

```

```

SummaryFile.write(",")
SummaryFile.write(row['STUDENT_FIRST'])
SummaryFile.write(",")
SummaryFile.write(str(LabHomeworkAverage))
SummaryFile.write(",")
SummaryFile.write(str(LabBonusPoint))
SummaryFile.write(",")
SummaryFile.write(str(LabQuizAverage))
SummaryFile.write(",")
SummaryFile.write(str(TestAverage))
SummaryFile.write(",")
SummaryFile.write(str(FinalTest))
SummaryFile.write(",")
SummaryFile.write(str(SemesterAverage))
SummaryFile.write(",")
SummaryFile.writelines(Grade)

```

```

#
# Debug_Routine
# It is used for testing and debugging
#
def Deubg_Routine(NumberOfTest, NumberOfRetake, NumberOfLabHomework,
NumberOfLabQuiz, NumberOfFinal, TestTotal, LabQuizTotal, TestAverage,
LabQuizAverage, FinalTest, SemesterAverage, Grade):
    print("<<<<< Debug Routine >>>>>")
    print("Number of Test",NumberOfTest)
    print("Number of Retake",NumberOfRetake)
    print("Number of Homework",NumberOfLabHomework)
    print("Number of Lab Quiz",NumberOfLabQuiz)
    print("Number of Final",NumberOfFinal)
    print()

    print("Test Array:",TestArray)
    print("Test Total: ", TestTotal)
    print("Quiz Lab Total: ", LabQuizTotal)
    print()

    print("Test Averages: ", TestAverage)
    print("Quiz Lab Averages: ", LabQuizAverage)
    print("Final Test: ", FinalTest)

    print("Semester Averages: ", SemesterAverage)
    print("Grade: ", Grade)

```

```

#
# CalculateTestTotal
# It sorts Test List and ignore the lowest test if retaken test exists in the
list.

```

```

#
def CalculateTestTotal(NumberOfTest, TestArray):
    # sort all tests with lowest in TestArray[0]
    TestArray.sort()
    TestTotal=0
    # ignore the worst test or TestArray[0] for tests total
    I=1
    while I <= NumberOfTest:
        TestTotal = TestTotal + TestArray[I]
        I = I + 1
    return(TestTotal)

#
# ReadDataFromTESTFile
#
def ReadDataFromTESTFile(LastName, FirstName):

    # Intitlize number of test, retake, lab homework, lab quize, & final
    NumberOfTest = 0
    NumberOfRetake = 0
    NumberOfLabHomework =0
    NumberOfLabQuiz = 0
    NumberOfFinal = 0

    # Intitlize total of test, lab quiz, lab homework, & lab quiz average
    LabQuizTotal = 0
    LabHomeworkTotal = 0
    LabQuizAverage = 0
    LabHomeworkAverage=0
    LabBonusPoint=0

    TestTotal = 0
    TestAverage = 0
    FinalTest = 0

    # empty test array set
    TestArray=[]

    with open(TEST_FILENAME) as TESTFile:
        reader = csv.DictReader(TESTFile)
        for Testrow in reader:

            if (LastName == Testrow['STUDENT_LAST']) and ( FirstName ==
Testrow['STUDENT_FIRST']):
                if DEBUG:
                    print(Testrow['SECTION_NO'], Testrow['STUDENT_LAST'],
Testrow['STUDENT_FIRST'],Testrow['TEST_TYPE'],
Testrow['TEST_NO'],Testrow['TEST_RESULT'])

```

```

        if (Testrow['TEST_TYPE']=="TEST"):
            NumberOfTest = NumberOfTest + 1
            TestArray.append(float(Testrow['TEST_RESULT']))
        if (Testrow['TEST_TYPE']=="RETAKE"):
            NumberOfRetake = NumberOfRetake + 1
            TestArray.append(float(Testrow['TEST_RESULT']))
        if (Testrow['TEST_TYPE']=="HOMEWORK"):
            NumberOfLabHomework = NumberOfLabHomework + 1
            LabHomeworkTotal = LabHomeworkTotal +
float(Testrow['TEST_RESULT'])
        if (Testrow['TEST_TYPE']=="QUIZ"):
            NumberOfLabQuiz = NumberOfLabQuiz + 1
            LabQuizTotal = LabQuizTotal + float(Testrow['TEST_RESULT'])
        if (Testrow['TEST_TYPE']=="FINAL"):
            NumberOfFinal = NumberOfFinal + 1
            FinalTest = float(Testrow['TEST_RESULT'])

# calculate lab homework avergae
if (NumberOfLabHomework!=0 and LabHomeworkTotal != 0 ):
    LabHomeworkAverage = LabHomeworkTotal / NumberOfLabHomework

# Calculate lab bonus point
if (NumberOfLabHomework!=0 and LabHomeworkTotal !=0):
    LabBonusPoint = Calculate_Lab_Bonus_Point
(LabHomeworkTotal/NumberOfLabHomework)

# calculate test total and test average
TestTotal = CalculateTestTotal(NumberOfTest, TestArray)
if (NumberOfTest !=0):
    TestAverage = TestTotal / NumberOfTest

# calculate lab quiz average
if (NumberOfLabQuiz !=0):
    LabQuizAverage = LabQuizTotal / NumberOfLabQuiz

# calculate semester average and grade
SemesterAverage = TestAverage * 0.60 + LabQuizAverage * 0.15 + FinalTest * 0.25

Grade = Calculate_Grade_Normal(SemesterAverage)

# Write data into summary report cvs file
WriteReportOneStudent(Testrow, LabHomeworkAverage, LabBonusPoint,
LabQuizAverage, TestAverage, FinalTest, SemesterAverage,Grade)

if DEBUG:
    Deubg_Routine(NumberOfTest, NumberOfRetake, NumberOfLabHomework,
NumberOfLabQuiz, NumberOfFinal, TestTotal, LabQuizTotal, TestAverage,
LabQuizAverage, FinalTest,SemesterAverage, Grade)

```

```

# Main Code Begin

# Set debug to True for testing and debugging
# DEBUG = not True
DEBUG = True

TestArray=[]

# Open summary file or report file
SummaryFile =open(SUMMARY__FILENAME,"w")
# Print header in the summary file
Print_Summary_File_Header()

# while there is a student in class thable do
#     read all records for the students from test file
#     do all calculation
# Wirte summary of student tests and etc. into summary report file.

with open(CLASS__FILENAME) as CLASSFile:
    reader = csv.DictReader(CLASSFile)
    for Classrow in reader:
        LastName = Classrow['STUDENT_LAST']
        FirstName = Classrow['STUDENT_FIRST']
        if (DEBUG):
            print(LastName, FirstName)
            ReadDataFromTESTFile(LastName, FirstName)

# close summary report file
SummaryFile.close()

# Main Code End

```